

LEXICAL ANALYZER AND TOKEN COUNTER

1. TITLE

Lexical Analyzer and Token Counter

2. OBJECTIVE

The objective of this project is to develop a Lexical Analyzer in C that reads a source-code file and identifies different types of tokens.

The program classifies tokens into Keywords, Identifiers, Operators, Constants, String Literals, Separators, Comments, and Special Symbols, and calculates the number of tokens in each category.

3. INTRODUCTION

A compiler is system software that translates a program written in a high-level programming language into a form that can be understood and executed by a computer.

Lexical Analysis is the first phase of a compiler. The component responsible for lexical analysis is called the Lexical Analyzer or Scanner.

The lexical analyzer reads the source program character by character and groups the characters into meaningful sequences called lexemes. Each lexeme is then classified into a token such as a keyword, identifier, operator, constant, or separator.

The output of lexical analysis is passed to the next phase of the compiler, which is syntax analysis.

4. PROBLEM STATEMENT

Develop a C program that performs lexical analysis on a given source-code file.

The program should:

Identify keywords.

Identify identifiers.

Identify operators.

Identify constants.

Identify string literals.

Identify separators.

Identify comments.

Identify special symbols.

Count the number of tokens in each category.

Display each token along with its corresponding type.

5. TOKEN TYPES

The lexical analyzer identifies the following token categories.

Keywords are reserved words that have predefined meanings in the C language.

Examples:

int

float

if

else

return

while

Identifiers are names given to variables, functions, arrays, and other program elements.

Examples:

sum

average

value

printf

Operators are symbols used to perform operations.

Examples:

+

-

*

/

=

>

<

==

Constants are fixed values that do not change during program execution.

6. ALGORITHM

Start the program.

Read the name of the source-code file.

Open the source-code file.

Read the file character by character.

Ignore white spaces and newline characters.

If the character is a letter or underscore, read the complete word.

Check whether the word is a keyword.

If it is a keyword, classify it as a Keyword; otherwise, classify it as an Identifier.

If the character is a digit, read the complete number and classify it as a Constant.

If double quotation marks are encountered, read the complete string and classify it as a String Literal.

Check for single-line and multi-line comments.

Identify operators and two-character operators.

Identify separators.

Identify remaining characters as Special Symbols.

Increment the corresponding token counter.

Display each token and its type.

Display the total count of each token category.

Stop the program.

7. SOURCE CODE

The source code for the project is stored in:

lexical_analyzer.c

8. SAMPLE INPUT

The following source program is used as the sample input:

```
int sum = a + b;
float average = sum / 2.0;
// Calculate average
if (average > 50)
    printf("Pass");
```

9. SAMPLE OUTPUT

TOKEN	TYPE

int	Keyword
sum	Identifier
=	Operator
a	Identifier
+	Operator
b	Identifier
;	Separator
float	Keyword
average	Identifier
=	Operator
sum	Identifier
/	Operator
2.0	Constant
;	Separator
// Calculate average Comment	

if	Keyword
(	Separator
average	Identifier
>	Operator
50	Constant
)	Separator
printf	Identifier
(	Separator
"Pass"	String Literal
)	Separator
;	Separator

TOKEN COUNT

Keywords : 3
 Identifiers : 7
 Operators : 5
 Constants : 2
 String Literals : 1
 Separators : 7
 Comments : 1
 Special Symbols : 0

11. TOKEN CLASSIFICATION

Token	Type
-------	------

int	Keyword
sum	Identifier
=	Operator
a	Identifier
+	Operator
b	Identifier
;	Separator
float	Keyword
average	Identifier
2.0	Constant

// Calculate average Comment

if Keyword

(Separator

averageIdentifier

> Operator

50 Constant

) Separator

printf Identifier

"Pass" String Literal

; Separator

10. TEST CASES

Test Case 1 – Keyword and Identifier

Input:

```
int x = 10;
```

Classification:

int → Keyword

x → Identifier

= → Operator

10 → Constant

; → Separator

Result: Accepted and correctly classified.

Test Case 2 – Floating Point Constant

Input:

```
float value = 25.5;
```

Classification:

float → Keyword
value → Identifier
= → Operator
25.5 → Constant
; → Separator

Result: Accepted and correctly classified.

Test Case 3 – String Literal

Input:

```
printf("Hello");
```

Classification:

printf → Identifier
(→ Separator
"Hello" → String Literal
) → Separator
; → Separator

Result: Accepted and correctly classified.

Test Case 4 – Comment

Input:

```
// This is a comment
```

Classification:

// This is a comment → Comment

Result: Accepted and correctly classified.

Test Case 5 – Operators

Input:

a = b + c;

Classification:

a → Identifier

= → Operator

b → Identifier

+ → Operator

c → Identifier

; → Separator

Result: Accepted and correctly classified.

11. CONCLUSION

The Lexical Analyzer and Token Counter was successfully implemented using the C programming language. The project demonstrates how the first phase of a compiler processes source code by identifying and classifying lexemes into different token types.

The program successfully recognizes keywords, identifiers, operators, constants, string literals, separators, comments, and special symbols and maintains their respective counts. This project provides a basic understanding of the working and importance of lexical analysis in compiler design.